

MODELLI E ARCHITETTURE AVANZATI DI BASI DI DATI

Alessandro Zappatore

Università degli studi di Torino
Anno accademico 2025/2026, 2° semestre

Contents

1	Introduzione alle Basi di Dati e ai Modelli Storici	3
1.1	Concetti Fondamentali	3
1.2	Livelli di Astrazione (Architettura Standard)	3
1.3	DBMS (Data Base Management System)	3
2	Carrellata Storica dei Modelli di Dati	5
2.1	Prima Generazione: Modelli Gerarchici e Reticolari (Anni '60)	5
2.2	Seconda Generazione: Modello Relazionale (Anni '70)	5
2.3	Terza Generazione: Anni '80 e il Modello ad Oggetti	6
2.4	Quarta Generazione: Anni '90 e 2000 - Dati Semistrutturati e Grafi	6
2.5	Modelli Ad-Hoc per Domini Specifici	7
3	Introduzione all'Architettura	8
3.1	Gestione dello Storage (Dati su Disco)	8
3.1.1	Organizzazione dei File	8
3.2	Gestione della Memoria e dei File	9
3.2.1	Disk Space Manager	9
3.2.2	Buffer Manager	9
3.3	Recovery Management	10
3.3.1	Recovery Manager (Gestore del Ripristino)	10
3.4	Concurrency Control	11
3.4.1	Lock Manager	11
3.4.2	Transaction Manager e Proprietà ACID	11
3.4.3	Dettagli sull'implementazione ACID e Log	12
3.5	Query Evaluation Engine	12
4	Dischi e Gestione della Memoria Secondaria	14
4.1	Architettura Fisica del Disco	14
4.2	Tempo di Accesso e Componenti di Costo	15
4.3	Strategie di Ottimizzazione e "Next Block"	15
4.4	Pre-fetching e Prevedibilità	15
4.5	File Manager e Organizzazione dei Dati	16
4.5.1	Strategie di Organizzazione dei File	16
5	Struttura Interna: Pagine e Record	18
5.1	Record	18
5.1.1	Record a Lunghezza Fissa (Fixed-Length)	18
5.1.2	Record a Lunghezza Variabile (Variable-Length)	18
5.2	Organizzazione di una pagina	18

5.2.1	Record a lunghezza fissa	19
5.2.2	Record a lunghezza variabile (Slotted Page)	19
5.3	Heap File	20
5.3.1	Heap File: Implementazione a Lista Concatenata	20
5.3.2	Heap File: Implementazione a Directory	21
6	Indici	22
6.1	Ricerca per intervallo (Range Search)	22
6.2	ISAM (Indexed Sequential Access Method)	22
6.3	Indici B+ Tree	24
6.3.1	Impatto della Dimensione della Chiave sulle Prestazioni	25
6.3.2	Compressione delle chiavi basata su prefissi (Prefix Key Compression)	26
6.3.3	Inizializzazione di un B+ Tree: Bulk Loading	27
6.4	Classificazione degli indici	27
6.5	Indici Clusterizzati e Non Clusterizzati	28
6.5.1	Indice Clusterizzato (<i>Clustered Index</i>)	29
6.5.2	Indice Non Clusterizzato (<i>Unclustered Index</i>)	30
6.5.3	Casi di Studio: Domande d'Esame	30
6.5.4	Gestione dei Duplicati e Struttura delle Voci (Data Entries)	31
6.6	Indici con chiavi di ricerca composte	31
6.6.1	Strategie di esecuzione per query multi-attributo	31
6.7	Indici Hash-Based	32
6.7.1	Hashing Statico	33

1 Introduzione alle Basi di Dati e ai Modelli Storici

1.1 Concetti Fondamentali

Database (Base di Dati) Un database è una collezione di dati organizzati in modo strutturato, in funzione del modello di riferimento scelto. È fondamentale che i dati siano organizzati in modo efficiente per poterne garantire una gestione e un'interrogazione ottimale (reperimento, analisi e visualizzazione univoca dei risultati).

Differenza tra Dato e Informazione

- **Dato:** È un elemento grezzo, privo di contesto (es. un numero, una stringa, una data).
- **Informazione:** È il dato interpretato e contestualizzato.

Esempio: Il numero "10" è un dato. Se lo associo al numero di studenti presenti a lezione, diventa un'informazione. Se lo associo al numero di libri che ho letto, diventa un'informazione diversa. Il dato di partenza è lo stesso, ma il contesto ne cambia il significato. Le basi di dati organizzano i dati proprio per rendere efficiente e rapida l'estrazione di queste informazioni.

Modello dei Dati È un formalismo matematico e rigoroso utilizzato per descrivere la struttura dei dati stessi (es. modello relazionale, a oggetti, a grafo). Ciascun modello esprime un insieme di vincoli intrinseci che l'organizzazione dei dati deve rispettare. Nel modello relazionale, ad esempio, i dati devono essere divisi in tabelle normalizzate, collegate tramite chiavi primarie ed esterne.

Schema Rappresenta l'applicazione di un modello di dati a un dominio specifico. Definisce un insieme di vincoli strutturali e di integrità. In un database relazionale, lo schema descrive le tabelle (relazioni), i campi (attributi), i tipi di dato e i vincoli (es. chiavi primarie ed esterne).

I vincoli sono essenziali per garantire l'**integrità dei dati** (es. evitare problemi di integrità referenziale se una chiave esterna punta a un record inesistente) e semplificano l'ottimizzazione e la gestione, sebbene limitino la flessibilità del modello per certi scopi.

1.2 Livelli di Astrazione (Architettura Standard)

L'organizzazione dei dati può essere analizzata su tre livelli principali:

1. **Livello Concettuale:** È il livello più alto e astratto. Descrive "cosa" deve essere rappresentato (la conoscenza del dominio) senza preoccuparsi dell'implementazione. Utilizza formalismi comprensibili agli utenti, come i diagrammi ER (Entità-Relazione) o UML.
2. **Livello Logico:** Si posiziona a metà strada e colma il gap tra concettuale e fisico. Descrive i dati in modo formale attraverso le strutture previste dal modello scelto (es. tabelle nel modello relazionale o gerarchie nel modello ad oggetti), ma rimane indipendente dalla loro effettiva memorizzazione fisica. Alcuni modelli logici (come quello ad oggetti) sono più vicini al concettuale, altri (come il relazionale) sono più vicini al livello fisico.
3. **Livello Fisico:** Descrive "come" i dati sono effettivamente memorizzati su disco. Riguarda le strutture dati interne, l'allocazione dello spazio, la codifica in bit (es. come rappresentare fisicamente una data) e gli indici.

1.3 DBMS (Data Base Management System)

Cos'è un DBMS È un sistema software (che talvolta include componenti hardware) progettato per creare, gestire, manipolare e interrogare una base di dati in modo efficiente e sicuro. L'obiettivo principale di un DBMS è garantire la separazione tra la "business logic" (le applicazioni) e i dati fisici. Per fare ciò, deve garantire due livelli di astrazione:

- **Indipendenza Logica dei Dati:** Le applicazioni non devono risentire di modifiche fatte allo schema logico (es. se aggiungo una colonna a una tabella, un programma che interrogava le altre colonne deve continuare a funzionare senza essere riscritto).
- **Indipendenza Fisica dei Dati:** Le applicazioni e lo schema logico non devono risentire di modifiche a livello fisico (es. se aggiungo o rimuovo un indice su un disco per velocizzare una ricerca, il programma esterno non se ne deve accorgere).

Differenza e Interazione tra OS e DBMS Un Sistema Operativo (OS) offre funzionalità di base per la gestione delle risorse hardware e software, come la gestione dei file, della memoria centrale (buffer), della concorrenza e il recupero da malfunzionamenti (crash recovery). Sebbene un DBMS si appoggi all'OS per l'accesso fisico alle risorse, esso implementa versioni *ad hoc* e altamente specializzate di queste funzionalità. I meccanismi generici dell'OS risultano infatti inadeguati per le esigenze specifiche di una base di dati.

Le differenze principali si manifestano in quattro macro-aree:

- **Gestione del Buffer (Buffer Management):** Nei database, l'operazione di gran lunga più costosa è l'I/O su disco. Entrambi i sistemi utilizzano un buffer in RAM per minimizzare le letture/scritture fisiche, ma adottano strategie di *replacement* profondamente diverse per scegliere la "vittima" (la pagina di memoria da sovrascrivere quando la RAM è piena). L'OS utilizza politiche generiche e reattive, come l'LRU (Least Recently Used), che scarta la pagina non utilizzata da più tempo. Questa politica può essere disastrosa per un DBMS: ad esempio, durante la scansione sequenziale di una tabella molto grande, l'LRU svuoterebbe l'intero buffer scartando dati potenzialmente utili per altre query. Il DBMS, invece, ha un comportamento **predittivo**. Grazie al modulo di ottimizzazione delle query (Query Optimizer), il DBMS conosce in anticipo il piano di esecuzione e sa quali dati serviranno a breve, potendo così forzare il mantenimento in memoria di pagine strategiche o scartare quelle inutili.
- **Gestione dei File e Strutture Dati:** Per l'OS, un file è semplicemente una sequenza (stream) di byte non strutturata. Per il DBMS, i dati hanno una struttura semantica precisa: file fisici suddivisi in blocchi, contenenti tuple, formate da attributi. Spesso il DBMS bypassa del tutto le routine di caching dei file dell'OS per avere il controllo assoluto su *quando* un blocco viene effettivamente scritto su disco.
- **Gestione della Concorrenza (Concurrency Control):** L'OS gestisce la concorrenza a livello di processi e thread. Il DBMS deve invece gestire la concorrenza a **livello logico del dato** (es. applicando lock temporanei su una singola riga di una tabella). Questo permette a migliaia di utenti simultanei di leggere e aggiornare gli stessi dati senza creare inconsistenze.
- **Recupero dai Guasti (Crash Recovery):** Mentre l'OS preserva la coerenza del file system, il DBMS deve garantire a livello logico l'integrità dei dati e il rispetto delle proprietà *ACID*. Tramite file di log e protocolli come il **Write-Ahead Logging (WAL)**, il DBMS può eseguire l'UNDO (annullare transazioni a metà) e il REDO (riapplicare transazioni completate) dopo un crash.

Transazione È una sequenza di operazioni trattata come un'unità logica e **atomica**: o tutte le operazioni vengono eseguite con successo, o nessuna viene applicata (principio di "tutto o niente"). *Esempio:* In un bonifico bancario (prelievo da un conto e deposito in un altro), se il deposito fallisce, il prelievo deve essere annullato per mantenere la consistenza ed evitare la perdita di denaro.

2 Carrellata Storica dei Modelli di Dati

2.1 Prima Generazione: Modelli Gerarchici e Reticolari (Anni '60)

Modello Gerarchico Basato su una struttura ad albero. La radice rappresenta il concetto principale e i nodi figli le sue proprietà o entità dipendenti (es. Radice: "Persona" → Figli: "Nome", "Data di Nascita"). I puntatori erano bidirezionali (padre-figlio e figlio-padre).

- **Pro:** Molto efficiente per relazioni strettamente gerarchiche.
- **Contro:** Estremamente limitante per relazioni complesse o molti-a-molti, dove un nodo dovrebbe avere "più padri".

Modello Reticolare (Codasyl) Estende il modello gerarchico trasformando l'albero in un grafo. Questo permetteva ai nodi di avere molteplici nodi padre e di esprimere le relazioni tramite archi.

Criticità della Prima Generazione

1. **Distanza dal modello concettuale:** Erano funzionali solo per app di natura procedurale, ma inadeguati per sistemi come ospedali o banche.
2. **Navigazione a Puntatori Fisici:** Si sapeva da dove partiva il puntatore ma non dove andava a finire nel disco fisicamente.
3. **Linguaggio fortemente Procedurale:** L'utente doveva essere un bravo programmatore, scrivendo la procedura esatta (il percorso dei puntatori) per raggiungere il dato (es. "Vai a Università → segui puntatore Studenti...").
4. **Imprevedibilità e scarsa ottimizzazione:** L'ottimizzatore non poteva prevedere il percorso di accesso (e quindi allocare la memoria o posizionare la testina del disco in modo efficiente), rendendo impossibile un'ottimizzazione automatica.

2.2 Seconda Generazione: Modello Relazionale (Anni '70)

Introdotta da E.F. Codd, è il modello tuttora più diffuso. I dati sono organizzati in **frammenti omogenei** (le relazioni, o tabelle). Ciascun record (riga) ha la stessa struttura formale degli altri.

Vantaggi principali:

- **Prevedibilità e Ottimizzazione:** L'omogeneità permette di stimare quante pagine di memoria bisognerà leggere. "Prevedibilità implica ottimizzabilità" (si sfruttano al meglio i movimenti fisici del disco).
- **Assenza di Puntatori:** Si usano **puntatori logici** (le chiavi esterne per fare JOIN) eliminando l'imprevedibilità del puntatore fisico.
- **Linguaggio Dichiarativo (SQL):** L'utente dice *cosa* vuole estrarre, e non *come* trovarlo. Sarà il DBMS a tradurlo nei passi fisici necessari.
- **Algebra Relazionale:** Bastano pochi operatori matematici per implementare le richieste SQL. Attenzione: l'SQL è dichiarativo, mentre l'algebra relazionale è procedurale. Dato che gli operatori sono pochi, ci sono meno combinazioni possibili per ottimizzare (anche se l'ottimizzazione resta un problema matematico NP-completo).
L'utente dichiara cosa vuole ottenere non come ottenerlo, ci penserà il DBMS a tradurre la richiesta in un piano di esecuzione efficiente.
- **Vicinanza al Modello Fisico:** Le righe piatte nelle tabelle sono facilmente mappabili in blocchi contigui su disco.

Svantaggi e Limiti del Modello Relazionale:

- **Dati Piatti ("Particellati"):** Per ricomporre un'informazione complessa bisogna eseguire costosi e numerosi JOIN.
- **Difficoltà con Strutture Complesse:** Non è possibile rappresentare nativamente l'ereditarietà (es. Persona → Studente/Docente) o gerarchie complesse.
- **Incapacità di Match Parziali:** Il relazionale restituisce risultati esatti. Se cerco dati per "somiglianza" (match parziali), il relazionale fallisce perché questa logica andrebbe implementata nell'applicativo esterno, violando il principio di indipendenza dei dati.

2.3 Terza Generazione: Anni '80 e il Modello ad Oggetti

Per superare i limiti del relazionale (soprattutto riguardo all'ereditarietà), si sono sviluppate due scuole di pensiero, entrambe basate sul concetto di *Oggetto*:

- **Object-Oriented (OO) - Spinta dall'Accademia:** Si abbandona totalmente il relazionale. Si creano basi di dati che memorizzano nativamente non solo i dati, ma anche i metodi (comportamenti). Nativamente supporta gerarchie ed ereditarietà. Si interrogano con linguaggi come l'OQL (Object Query Language).
- **Object-Relational (OR) - Spinta dall'Industria:** Abbandonare il relazionale era economicamente folle visto il suo successo. Nasce così un approccio "ibrido": un motore relazionale di base arricchito da funzionalità ad oggetti utilizzabili solo "se necessarie", mantenendo la compatibilità con i vecchi sistemi.

2.4 Quarta Generazione: Anni '90 e 2000 - Dati Semistruutturati e Grafi

Con l'avvento di Internet, i database non sono più "isole" chiuse in una singola azienda, ma devono comunicare. Scambiarsi tabelle relazionali era problematico per colpa degli schemi rigidi.

Basi di Dati Semistructurate (XML) Si cercava flessibilità (es. in un DB l'indirizzo è un campo unico, in un altro è diviso in via/civico). Nasce l'esigenza di documenti **autodescrittivi** come l'XML. In questi sistemi, **lo schema stesso diventa interrogabile**. *Esempio:* "Dimmi se esiste il campo 'Indirizzo' in questo nodo, e se sì, dammi il suo valore". Anche qui vinse un approccio ibrido: esistono DB nativamente XML, ma i giganti dell'industria hanno integrato il supporto XML sopra i propri DB Relazionali.

Database a Grafo (dal 2003) A differenza degli anni '60 in cui si forzava la struttura a grafo per tutto, ora i grafi vengono usati *esclusivamente* per domini in cui i concetti sono naturalmente legati da complesse reti di relazioni.

- **Web Semantico e RDF:** Permette di descrivere la conoscenza tramite **triplette (Soggetto, Predicato, Oggetto)**. Lo schema non è rigido né definito a priori (es. "Sapino" → gestisce → "Corso"). Posso effettuare **query di raggiungibilità** che nei modelli precedenti erano impossibili (es. "Trova il percorso relazionale più breve che collega il prof. X al dipartimento Y"). Si usa il linguaggio dichiarativo SPARQL.
- **Social Network:** Simili all'RDF ma con milioni di nodi (persone) e poche tipologie di archi (es. "Follow", "Like"). Se lo facessimo nel modello relazionale servirebbero infiniti JOIN. I DB a grafo permettono di calcolare facilmente il ruolo di un nodo (es. la "centralità" di un influencer) o di ricercare pattern ricorrenti (usati moltissimo ad esempio in biologia per studiare le catene di proteine).

2.5 Modelli Ad-Hoc per Domini Specifici

Oggi si è compreso che un "modello universale" non basta per tutto, e sono nati sistemi specializzati:

- **Database Spaziali (GIS):** L'elemento chiave è la *Location* (coordinate spaziali). Si usano per query impossibili in SQL tradizionale, come le *query di range / Nearest Neighbours* ("Trova i ristoranti entro 5km da me") e il **Join Spaziale** ("Abbina le scuole del comune con i supermercati più vicini a ciascuna di esse").
- **Database Documentali e Multimediali:** Permettono query basate sulla **somiglianza** e non sull'esatta uguaglianza (es. "Trova profili di lavoratori simili al mio").
- **Gestione dell'Incertezza:** I DB relazionali operano sotto l'assunzione di mondo chiuso (tutto ciò che è nel DB è vero al 100%, ciò che non c'è è falso). Alcuni modelli più moderni permettono di associare pesi di probabilità (es. "Lo studente X frequenta il corso Y con l'80% di confidenza").

3 Introduzione all'Architettura

Un Database Management System (DBMS) è un sistema software complesso suddiviso in moduli funzionali. L'architettura è tipicamente a livelli, dove ogni livello offre servizi a quello superiore nascondendo i dettagli implementativi del livello inferiore.

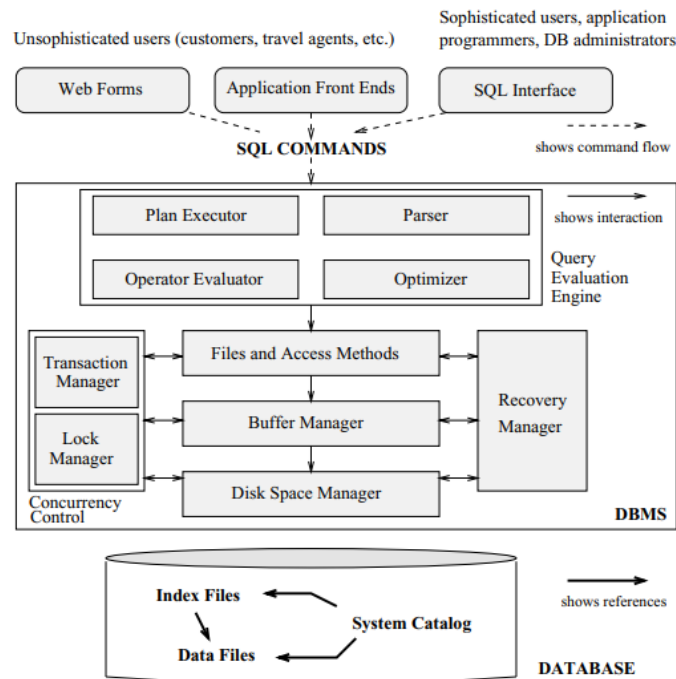


Figure 1: Panoramica dell'architettura a livelli di un DBMS

3.1 Gestione dello Storage (Dati su Disco)

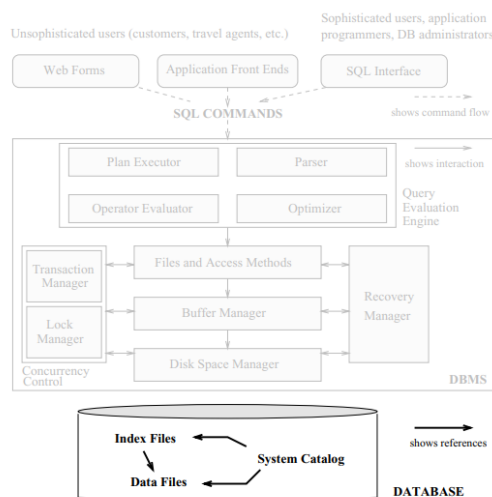


Figure 2: Rappresentazione dei dati su disco

Alla base del DBMS vi è la gestione fisica dei dati. I database risiedono su memoria secondaria (disco rigido o SSD) poiché la memoria volatile (RAM) non garantisce la persistenza e non è sufficientemente capiente per database di grandi dimensioni.

3.1.1 Organizzazione dei File

I dati su disco sono organizzati in file. A livello logico, il DBMS distingue tre categorie principali di file:

- **Data Files:** Contengono i dati veri e propri (i record delle tabelle). Possono essere organizzati come:
 - *Heap Files:* File non ordinati (mucchio), dove i nuovi record vengono inseriti nell'ultimo spazio disponibile. È la struttura più generica e veloce in scrittura.
 - *Sorted Files:* File in cui i record sono ordinati fisicamente in base a una chiave di ricerca. Ottimi per le letture sequenziali o ricerche su range.
- **Index Files:** Strutture ausiliarie (spesso alberi B+ o Hash) che velocizzano il recupero dei dati.
 - Contengono coppie (*chiave di ricerca, puntatore*).
 - Il puntatore indica l'indirizzo fisico (RID - Record ID) dove si trova il record completo nel data file.
- **System Catalogs (Dizionario dei Dati):** Contengono i metadati, ovvero "dati sui dati". Includono:
 - Schemi delle tabelle, nomi degli attributi e tipi di dato.
 - Statistiche per l'ottimizzatore (es. numero di record, cardinalità, distribuzione dei valori).
 - Informazioni sugli utenti e sui permessi di accesso.

3.2 Gestione della Memoria e dei File

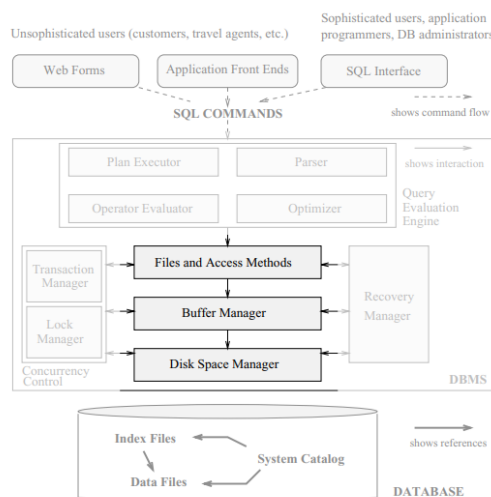


Figure 3: Interazione tra Disk Space Manager e Buffer Manager

Questo livello fa da "ponte" tra il disco (lento e persistente) e il processore (veloce e volatile).

3.2.1 Disk Space Manager

È il modulo di più basso livello che interagisce direttamente con il File System del Sistema Operativo.

- Gestisce l'allocazione e la deallocazione dello spazio su disco.
- Astrazione: Nasconde i dettagli hardware (tracce, settori) offrendo ai livelli superiori il concetto di **Pagina** (blocco di dati di dimensione fissa, es. 4KB o 8KB).

3.2.2 Buffer Manager

È uno dei componenti più critici per le prestazioni. Gestisce la memoria principale (RAM) riservata al DBMS, denominata **Buffer Pool**.

Funzionamento L'utente o i moduli superiori non accedono mai direttamente al disco. La procedura è la seguente:

1. **Richiesta:** Un livello superiore richiede una pagina.
2. **Check:** Il Buffer Manager controlla se la pagina è già nel Buffer Pool.
3. **Hit o Miss:**
 - Se presente (*Cache Hit*): restituisce l'indirizzo in memoria.
 - Se assente (*Cache Miss*): carica la pagina dal disco in un frame libero del buffer.

Strategie e Concetti Chiave

- **Unità di trasferimento:** Si legge/scrive sempre un'intera pagina, mai un singolo record. Questo sfrutta il principio di *località spaziale* (è probabile che dati vicini servano a breve).
- **Località temporale:** Se una pagina viene richiesta, è probabile che venga richiesta nuovamente a breve.
- **Page Replacement:** Quando il buffer è pieno, il gestore deve decidere quale pagina "scaricare" per fare spazio a quella nuova. Politiche comuni sono LRU (*Least Recently Used*) o Clock.
- **Dirty Page:** Una pagina modificata in memoria ma non ancora scritta su disco è detta "sporca". Viene scritta su disco solo quando necessario per minimizzare l'I/O (scrittura pigra o *lazy*).

3.3 Recovery Management

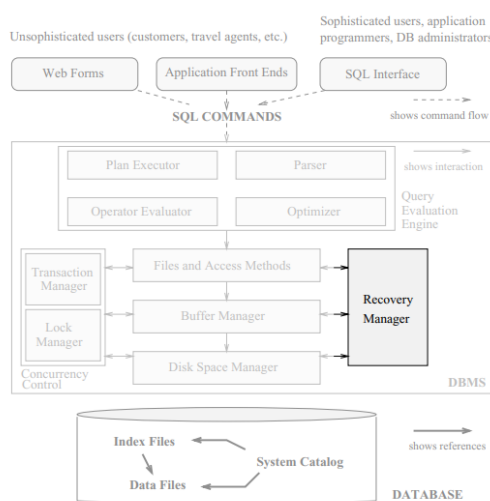


Figure 4: Recovery Manager e gestione dei crash

Questo livello garantisce l'integrità del database e la gestione degli accessi concorrenti tramite due moduli che collaborano strettamente.

3.3.1 Recovery Manager (Gestore del Ripristino)

Il suo obiettivo è garantire la durabilità e l'atomicità, proteggendo i dati da guasti (crash di sistema, blackout).

Disallineamento RAM/Disco Per efficienza, il Buffer Manager ritarda le scritture su disco. In caso di crash, il contenuto della RAM viene perso. Il Recovery Manager risolve questo problema utilizzando tecniche come il **WAL (Write-Ahead Logging)**: ogni modifica viene scritta in un file di Log (sicuro e sequenziale) *prima* che la modifica venga applicata al database vero e proprio.

Operazioni di Ripristino Al riavvio dopo un crash, il sistema analizza il Log ed esegue:

- **Undo (Annullamento):** Annulla le modifiche di transazioni che erano attive ma non completate al momento del crash (garantisce l'Atomicità).
- **Redo (Rifacimento):** Ripplica le modifiche di transazioni che erano state confermate (commit) ma le cui pagine di dati non erano ancora state scritte su disco (garantisce la Durabilità).

Esempio: In un bonifico bancario, se i soldi vengono tolti dal conto A (operazione 1) ma il sistema crasha prima di aggiungerli al conto B (operazione 2), il Recovery Manager tramite *Undo* annulla il prelievo su A, riportando il database a uno stato consistente.

3.4 Concurrency Control

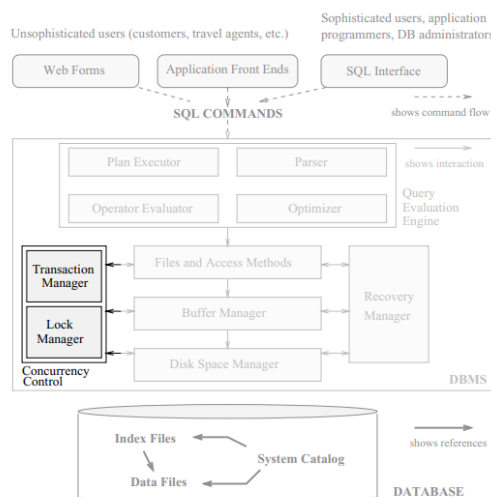


Figure 5: Concurrency Control e gestione dei Lock

3.4.1 Lock Manager

Gestisce l'accesso simultaneo di più utenti ai dati per evitare conflitti. Il modulo principale è il **Lock Manager**, che assegna permessi di accesso:

- **Lock Condiviso (S):** Permette la lettura. Più transazioni possono detenere un lock S sullo stesso oggetto.
- **Lock Esclusivo (X):** Permette la scrittura (e lettura). Solo una transazione alla volta può averlo.

I lock possono avere diversa *granularità*: Database, Tabella, Pagina o Record. Una granularità fine (record) aumenta la concorrenza ma anche l'overhead di gestione.

Deadlock Un rischio dei lock è lo stallo (*deadlock*): Transazione A aspetta una risorsa bloccata da B, mentre B aspetta una risorsa bloccata da A. Il Lock Manager deve rilevare e risolvere queste situazioni, solitamente abortendo una delle due transazioni.

3.4.2 Transaction Manager e Proprietà ACID

Il Transaction Manager coordina le operazioni per garantire che un insieme di azioni venga trattato come un'unica unità logica. Deve soddisfare le proprietà **ACID**:

- **A - Atomicità (All-or-Nothing):** La transazione è indivisibile. O vengono eseguite tutte le operazioni, o nessuna. In caso di errore, si esegue il *Rollback*.
- **C - Consistenza:** La transazione deve portare il DB da uno stato valido a un altro stato valido, rispettando tutti i vincoli di integrità (es. chiavi primarie, vincoli check).

- **I - Isolamento:** L'esecuzione concorrente di più transazioni non deve interferire l'una con l'altra. Ogni transazione deve "credere" di essere l'unica in esecuzione.
- **D - Durabilità:** Una volta che una transazione ha fatto *Commit* (conferma), le sue modifiche sono permanenti e non andranno perse, nemmeno in caso di crash catastrofico.

3.4.3 Dettagli sull'implementazione ACID e Log

Per garantire l'efficienza, il **Transaction Manager** gestisce un **File di Log** dove registra le operazioni per eventuale undo/redo. Tuttavia, per non rallentare il sistema, anche il log viene inizialmente scritto in memoria volatile. Questo introduce la necessità di protocolli avanzati per garantire che, al momento del *commit*, i dati siano effettivamente persistenti (Durabilità) nonostante il passaggio in RAM.

Le responsabilità delle proprietà ACID sono così ripartite:

- **Atomicità e Durabilità:** Gestite dal **Recovery Manager** (tramite log e ripristino).
- **Isolamento e Consistenza:** Gestite dal **Concurrency Control** (cooperazione tra Lock Manager e Transaction Manager).

Nota sul Locking: Esiste un trade-off fondamentale. Un locking molto rigido previene errori e dead-lock, ma riduce il *throughput* (transazioni per secondo) impedendo la parallelizzazione. I DBMS moderni cercano un equilibrio per massimizzare le prestazioni senza compromettere l'integrità.

Precisazione sull'Ottimizzatore L'**Optimizer** non cerca sempre la soluzione ottima in senso assoluto, poiché calcolarla richiederebbe troppo tempo. Utilizza invece delle **euristiche** per scartare i piani pessi e sceglierne uno "sufficientemente buono" in tempi rapidi.

3.5 Query Evaluation Engine

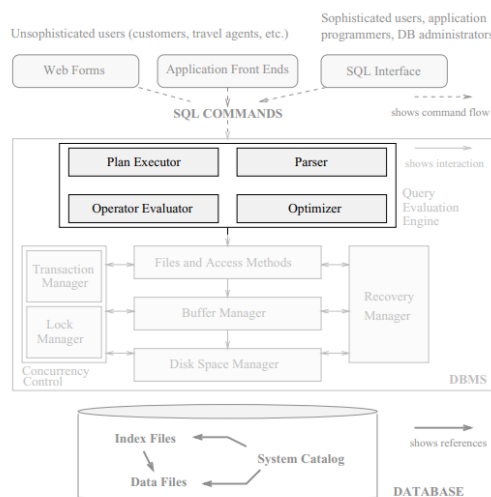


Figure 6: Flusso di esecuzione di una Query

È la parte "intelligente" del DBMS, responsabile dell'elaborazione delle richieste utente (solitamente in SQL).

Il processo si divide in quattro fasi principali:

1. Parser (Analizzatore):

- Verifica la correttezza sintattica (es. SQL scritto bene).
- Verifica la semantica (es. le tabelle e colonne esistono? L'utente ha i permessi?).
- Traduce la query in un albero di espressione dell'algebra relazionale.

2. Query Optimizer (Ottimizzatore):

- È il cervello del sistema. Per una stessa query esistono molti modi di eseguirla (piani di esecuzione).
- Utilizza le statistiche del *System Catalog* per stimare il costo (I/O, CPU) di ogni piano.
- Sceglie il piano di esecuzione più efficiente (es. decide se usare un indice o scansionare tutta la tabella).

3. Plan Executor:

- Prende il piano scelto dall'ottimizzatore e lo esegue, coordinando le operazioni.

4. Operator Evaluator:

- Implementa i singoli algoritmi per le operazioni relazionali (Join, Select, Project, Sort). Interagisce con il Buffer Manager per recuperare le pagine necessarie.

4 Dischi e Gestione della Memoria Secondaria

Il DBMS memorizza le informazioni su dischi magnetici (*hard disks*), dispositivi di memoria secondaria che garantiscono la **persistenza** dei dati: a differenza della RAM (volatile), i dati su disco vengono mantenuti anche dopo lo spegnimento del server.

Le implicazioni principali per il design di un DBMS derivano dalla differenza di velocità tra memoria centrale e disco:

- **Costo delle operazioni:** Le operazioni di I/O (Input/Output), ovvero *Read* (trasferimento Disco → RAM) e *Write* (trasferimento RAM → Disco), sono estremamente costose in termini di tempo rispetto alle operazioni in memoria centrale.
- **Obiettivo:** Il DBMS deve essere progettato per minimizzare il numero di accessi al disco, poiché sono il vero collo di bottiglia del sistema.

4.1 Architettura Fisica del Disco

Un disco rigido è costituito da una pila di piatti (superfici circolari magnetiche) che ruotano continuamente attorno a un perno centrale.

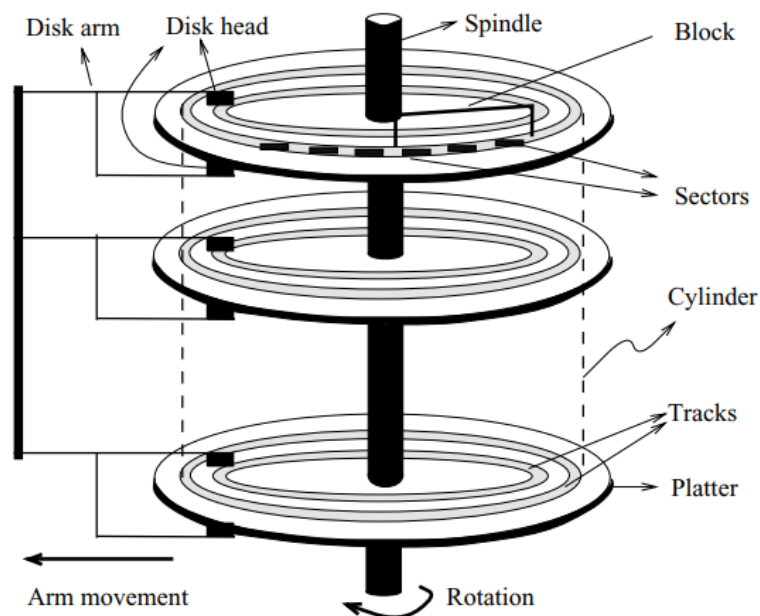


Figure 7: Struttura di un disco rigido: piatti, testine, braccio meccanico e cilindri.

L'organizzazione fisica è la seguente:

- **Piatti e Tracce:** I dati sono memorizzati su cerchi concentrici chiamati **tracce**.
- **Testine:** La lettura/scrittura avviene tramite testine montate su un unico braccio meccanico (attuatore). Le testine si muovono solidalmente: non possono accedere a punti diversi indipendentemente.
- **Cilindro:** L'insieme delle tracce che si trovano alla stessa distanza dal centro su tutti i piatti impilati forma un cilindro.
- **Settori:** Ogni traccia è divisa in settori. Un blocco logico (o **pagina**) corrisponde tipicamente a uno o più settori fisici ed è l'unità minima di lettura/scrittura (non è possibile leggere mezzo record, si legge sempre l'intera pagina).

4.2 Tempo di Accesso e Componenti di Costo

L'accesso ai dati su disco non è uniforme (come nella RAM, che è *Random Access Memory*), ma dipende dalla posizione fisica dei dati e dal movimento meccanico. Il tempo totale di accesso è la somma di tre componenti:

- **Seek Time** (1-20 ms): È il tempo necessario per spostare il braccio meccanico in modo che la testina si posizioni sulla traccia corretta.
 - È la componente più costosa (movimento meccanico macroscopico).
 - Dipende dalla distanza tra la posizione attuale della testina e quella di destinazione. Se i dati sono sulla stessa traccia, il seek time è nullo.
- **Rotational Latency** (0-10 ms): Una volta posizionata la testina sulla traccia, bisogna attendere che il disco ruoti finché il settore desiderato non passa sotto la testina.
 - Nel caso peggiore serve un giro completo; nel caso medio, mezzo giro.
 - I dischi ruotano costantemente per evitare l'inerzia di stop-and-go.
- **Transfer Time**: È il tempo necessario per il passaggio effettivo dei dati dalla superficie magnetica alla memoria principale (lettura vera e propria). È molto veloce rispetto ai primi due.

4.3 Strategie di Ottimizzazione e "Next Block"

Poiché il Seek Time è la fase più lenta, l'obiettivo principale nella memorizzazione dei file è massimizzare l'**accesso sequenziale** e minimizzare quello casuale (*random*).

Si introduce il concetto logico di **Next Block**: se un file occupa più pagine, il DBMS deve decidere dove allocare la pagina successiva per minimizzare i tempi di lettura futuri.

L'ordine di preferenza per l'allocazione ("Next Block") è:

1. **Stessa Traccia**: La soluzione ideale. Nessun movimento del braccio (Seek = 0) e attesa di rotazione minima.
2. **Stesso Cilindro**: Se la traccia è piena, si usa la traccia corrispondente su un altro piatto (stesso cilindro). Richiede solo il cambio elettronico della testina attiva, ma nessun movimento meccanico del braccio.
3. **Cilindro Adiacente**: Se il cilindro è pieno, si passa a quello fisicamente più vicino per ridurre al minimo il Seek Time.

4.4 Pre-fetching e Prevedibilità

Sfruttando il principio di località spaziale e l'organizzazione sequenziale (Next Block), il sistema può applicare il **Pre-fetching**:

- Quando viene richiesta una pagina, il sistema intuisce (prevedibilità) che probabilmente serviranno anche le pagine successive.
- Invece di fermare la lettura, il disco continua a leggere e carica in memoria (nel Buffer) anche le pagine adiacenti che passano sotto la testina.
- Questo trasforma molteplici potenziali richieste di I/O future in un'unica operazione sequenziale attuale, abbattendo drasticamente i costi.

4.5 File Manager e Organizzazione dei Dati

Il **File Manager** è il componente del DBMS responsabile della gestione dello spazio su disco e delle strutture dati utilizzate per memorizzare le informazioni. Possiamo classificare i file in due macro-categorie principali:

- **File di Dati:** Contengono i record effettivi delle tabelle (il contenuto del database). Possono essere organizzati come *Heap File* (non ordinati) o *Sorted File* (ordinati).
- **File di Indice:** Strutture ausiliarie (es. B+ Tree, Hash) che velocizzano l'accesso ai dati. Non contengono i dati completi, ma associano una chiave di ricerca alla posizione fisica del dato.

Fondamentale per il collegamento tra questi file è il concetto di **RID (Record ID)**.

RID (Record Identifier): È l'identificatore univoco e invariante di un record. Rappresenta l'indirizzo fisico o logico che permette al DBMS di recuperare esplicitamente un record specifico.

4.5.1 Strategie di Organizzazione dei File

La scelta di come organizzare i file su disco influenza drasticamente le prestazioni in lettura e scrittura. Non esiste una soluzione ottima universale, ma diverse alternative:

1. Heap Files (File Non Ordinati) È l'organizzazione più semplice. I record sono accumulati nel file senza un ordine specifico (spesso in ordine di arrivo/inserimento).

- **Inserimento:** Molto efficiente ($O(1)$). I nuovi dati vengono semplicemente accodati all'ultima pagina disponibile (*append-only*).
- **Ricerca:** Inefficiente ($O(N)$). Per trovare un record specifico è necessario scansionare l'intero file (linear scan), leggendo tutte le pagine.

2. Sorted Files (File Ordinati) I record sono memorizzati fisicamente in ordine sequenziale in base al valore di uno o più campi (*chiave di ordinamento*), nel caso di più campi l'ordine si fa su tutti i campi insieme non per il singolo. Un sorted files può essere ordinato per una **singola chiave di ordinamento!**

In presenza di più chiavi di ordinamento, devo scegliere una sola, solitamente quella più frequentemente usata nelle query di ricerca. Essendo i file solitamente memorizzati in modo contiguo, si prestano per la ricerca binaria che ha costo logaritmo in base al numero di record, scegliere come chiave di ordinamento un campo con un grande carico di lavoro di ricerca è fondamentale per le prestazioni. Es. `SELECT * WHERE cognome = "Rossi"`, se il file non è ordinato devo scansionare tutti i record, se è ordinato per `Cognome` posso usare la ricerca binaria e trovare tutti i record con cognome "Rossi" in modo efficiente e appena ne trovo uno diverso mi posso fermare perché sono certo che non ne troverò altri più avanti.

Es. se l'ordinamento è su (`Cognome`, `Nome`), i record sono ordinati prima per `Cognome` e in caso di parità per `Nome`.

- **Ricerca:** Molto efficiente. È possibile utilizzare la **Ricerca Binaria**, che ha costo $O(\log_2 N)$. Anche senza un indice, si accede alla pagina centrale e ci si sposta a destra o sinistra.
- **Manutenzione (Scrittura/Modifica):** Costosa. Per mantenere l'ordinamento fisico:
 - Bisogna trovare la posizione corretta per il nuovo record.
 - Se non c'è spazio nella pagina, è necessario effettuare uno **shifting** (spostamento) di tutti i record successivi per creare lo spazio. Questo può propagarsi a cascata sulle pagine seguenti.
- **Strategia del Buffer (Fill Factor):** Per mitigare il costo dello shifting, le pagine non vengono mai riempite al 100%. Si lascia una porzione di spazio libero (es. 20%) in ogni pagina per accomodare futuri inserimenti o espansioni dei record senza dover spostare dati tra pagine diverse.

3. Index Files (Indici) Sono strutture dati progettate per fornire **accesso diretto** ai record senza scansionare il file dati.

- Associano una *Chiave di Ricerca* a un *RID*.
- **Tipi comuni:**
 - *Hash Index*: Ottimi per ricerche di uguaglianza esatta (es. $ID = 5$).
 - *Tree Index* (es. *B+ Tree*): Ottimi per ricerche per intervallo (es. $Et\grave{a} > 18$).
- **Trade-off**: Gli indici occupano spazio aggiuntivo su disco e rallentano le operazioni di scrittura (poiché a ogni INSERT o DELETE nella tabella bisogna aggiornare anche l'indice), ma velocizzano drasticamente le letture.

5 Struttura Interna: Pagine e Record

La **pagina** è l'unità atomica di trasferimento I/O tra disco e memoria. Un file è una sequenza di pagine. All'interno delle pagine risiedono i **record**. Esistono due approcci principali per la rappresentazione dei record:

5.1 Record

5.1.1 Record a Lunghezza Fissa (Fixed-Length)

In fase di schema design, si stabilisce una dimensione fissa in byte per ogni attributo (es. CHAR(20), INTEGER). Tutti i record della tabella avranno la stessa dimensione totale.

- **Vantaggi (Prevedibilità):**

- L'accesso ai campi è immediato tramite aritmetica dei puntatori. Se so che i primi due campi occupano 10 byte totali, il terzo campo inizierà sempre al byte 11.
- Non serve memorizzare metadati sulla lunghezza.

- **Svantaggi:**

- **Frammentazione Interna (Spreco):** Se un campo CHAR(100) contiene solo "Ciao", vengono sprecati 96 byte. I dati nulli occupano comunque spazio fisico.
- Sprecare spazio significa memorizzare meno record per pagina, aumentando il numero di I/O necessari per leggere la tabella.

5.1.2 Record a Lunghezza Variabile (Variable-Length)

Necessari quando i dati hanno lunghezze eterogenee (es. VARCHAR, descrizioni testuali). Il record si adatta al contenuto effettivo.

Problematiche: viene meno la prevedibilità posizionale. Non so a priori dove inizia il terzo campo senza sapere quanto sono lunghi i primi due.

Esistono due tecniche implementative principali:

1. Uso di Separatori (Delimiters)

- I campi sono separati da un carattere speciale (es. |, \$).
- **Gestione NULL:** efficiente, basta mettere due separatori adiacenti (||).
- **Svantaggi:** accesso lento. Per leggere l'ultimo campo devo scansionare byte per byte tutto il record cercando i separatori. Inoltre, se il carattere separatore compare nei dati, deve essere gestito con sequenze di escape, aumentando la complessità.

2. Directory di Puntatori (Record Header) [Approccio consigliato]

- All'inizio del record (testa) viene inserita una piccola "directory" che contiene i puntatori (o *offset*) relativi alla fine di ogni campo variabile.
- **Vantaggi:**
 - **Accesso $O(1)$:** per leggere il campo *X*, leggo l'offset nell'header e salto direttamente a quella posizione di memoria, senza scansionare il resto.
 - **Overhead ridotto:** lo spazio occupato dagli offset è minimo rispetto allo spreco dei separatori o dei record fissi.
- **Vincoli:** la dimensione in bit riservata per i puntatori nell'header determina la dimensione massima teorica del record (es. se uso 16 bit per l'offset, il record non può superare 64KB).

5.2 Organizzazione di una pagina

La pagina è l'unità atomica di I/O. La sfida principale è come organizzare i record al suo interno per massimizzare lo spazio e gestire efficientemente inserimenti e cancellazioni.

5.2.1 Record a lunghezza fissa

In questo scenario, tutti i record hanno la stessa dimensione L . La pagina può essere vista come un array di slot di dimensione L .

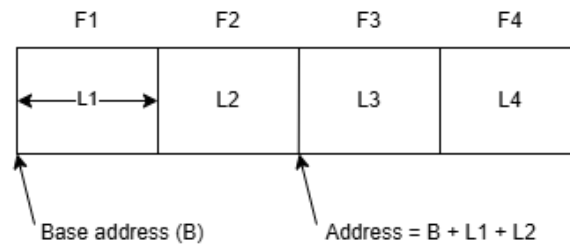


Figure 8: Record a lunghezza fissa

L'informazione essenziale da mantenere nell'header della pagina è il numero di record attualmente presenti. Esistono due strategie principali di gestione:

1. **Gestione Packed (Contigua):** i record occupano sempre le prime N posizioni (slot) della pagina senza lasciare buchi.
 - **Inserimento:** molto veloce, si scrive nel primo slot libero alla fine.
 - **Cancellazione:** quando si rimuove un record, si crea un "buco". Per mantenere la contiguità (packed), si sposta l'ultimo record della pagina nel buco appena creato (oppure si traslano tutti i successivi, ma è costoso).
 - **Problema dei RID:** spostando i record per colmare i buchi, il loro indirizzo fisico cambia. Questo invalida tutti i puntatori esterni (indici) che puntavano a quel record tramite il RID. È un approccio sconsigliato se ci sono indici esterni.
2. **Gestione Unpacked (Bitmap):** i record vengono inseriti nel primo slot libero, ma non vengono mai spostati dopo l'inserimento. I buchi rimangono vuoti finché non viene inserito un nuovo record.
 - **Gestione:** l'header della pagina contiene una **Bitmap** con un bit per ogni slot (1 = pieno, 0 = vuoto).
 - **Vantaggio (RID stabili):** poiché i record non si spostano mai, il RID (costituito da *PageID* + *SlotNumber*) rimane valido per tutta la vita del record. Non è necessario aggiornare gli indici esterni in caso di cancellazione di altri record nella stessa pagina.

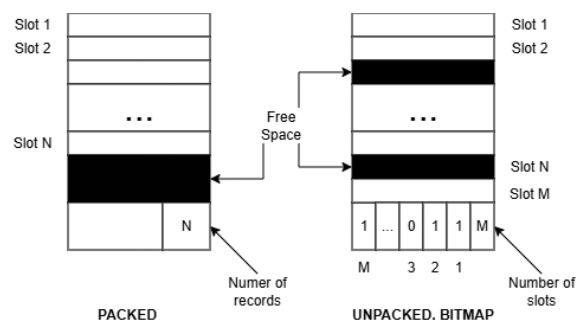


Figure 9: Organizzazione di una pagina con record a lunghezza fissa (con Bitmap)

5.2.2 Record a lunghezza variabile (Slotted Page)

Quando i record hanno lunghezza variabile, non possiamo dividere la pagina in slot fissi. La struttura standard utilizzata dai DBMS è la **Slotted Page Organization**.

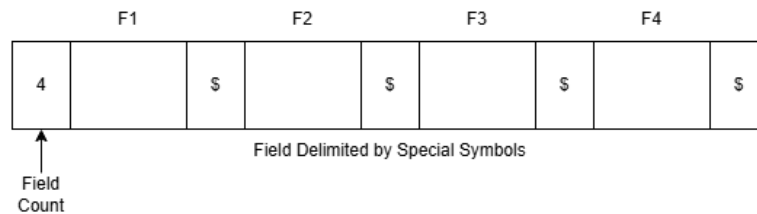


Figure 10: Record a lunghezza variabile

La gestione dello spazio libero diventa critica: un buco lasciato da un record cancellato di 20 byte non può ospitare un nuovo record di 30 byte (frammentazione).

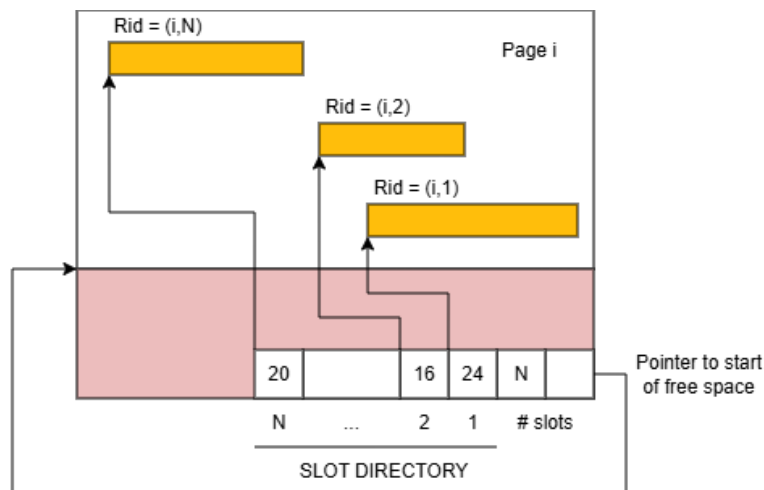


Figure 11: Slotted Page: Organizzazione di una pagina con record a lunghezza variabile

La struttura tipica prevede:

- **Footer/Directory (in basso):** contiene un array di slot. Ogni slot contiene una coppia $\langle \text{Offset}, \text{Lunghezza} \rangle$ che punta all'inizio del record corrispondente all'interno della pagina.
- **Dati (in alto):** i record vengono inseriti contigualmente dall'inizio della pagina verso il fondo.
- **Spazio Libero (al centro):** si trova tra l'ultimo record inserito e l'inizio della directory degli slot.

Compattazione e Indirizione: se un record viene cancellato, si crea un buco. Il sistema può spostare i record adiacenti per compattare lo spazio libero ("de-frammentazione"). Grazie alla directory degli slot, quando spostato un record fisicamente, aggiorniamo solo il suo **offset** nel footer. Il RID esterno punta all'indice dello slot (che non cambia), non all'indirizzo fisico del byte. Questo garantisce che i puntatori esterni rimangano validi anche se i dati si muovono all'interno della pagina.

5.3 Heap File

5.3.1 Heap File: Implementazione a Lista Concatenata

Un Heap File è un insieme di pagine senza ordinamento. Una possibile implementazione usa liste doppiamente concatenate gestite tramite una pagina header.

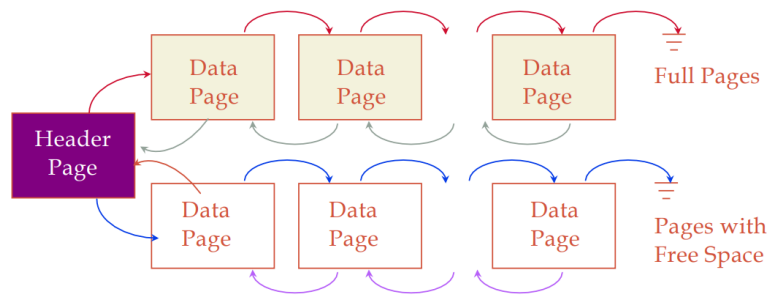


Figure 12: Organizzazione di un heap file come lista

Il DBMS mantiene due liste distinte:

1. **Full Pages List:** lista delle pagine completamente piene (o con spazio insufficiente per nuovi record). Non vengono consultate durante l'inserimento.
2. **Free Space List:** lista delle pagine che hanno almeno un po' di spazio libero.

Funzionamento:

- Per inserire un record, si scorre la lista delle pagine con spazio libero. Se la prima pagina ha spazio sufficiente, si inserisce. Se diventa piena, viene spostata nella lista delle "Full Pages".
- **Svantaggio:** se cerco spazio per un record molto grande, potrei dover scorrere molte pagine della "Free List" che hanno spazio libero ma insufficiente, generando I/O inutile.

5.3.2 Heap File: Implementazione a Directory

Questa è l'implementazione più efficiente e diffusa.

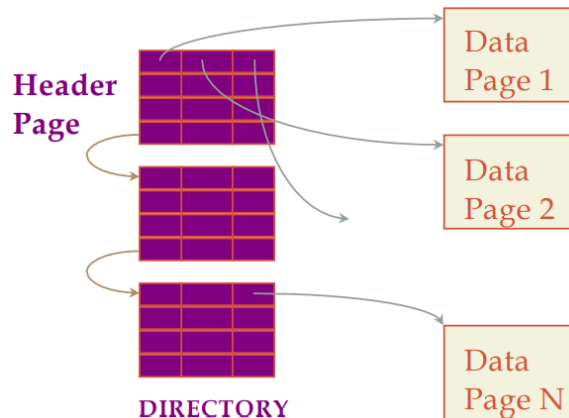


Figure 13: Organizzazione di un heap file usando una directory

Invece di liste, si usa una **Directory Page** (o un insieme di pagine directory) che mappa lo stato di ogni pagina del file dati.

Ogni voce della directory contiene:

- Puntatore alla pagina (PageID).
- **Free Space Count:** numero di byte liberi in quella pagina (o un indicatore approssimativo, es. 2 bit per indicare vuota, mezza piena, piena).

Vantaggi:

- La directory è molto compatta (una pagina di directory gestisce migliaia di pagine dati).
- Per inserire un record, il DBMS consulta la directory (in RAM o con 1 I/O) e trova subito una pagina con spazio sufficiente, senza dover caricare e controllare le pagine dati una per una.

6 Indici

Gli **indici** sono strutture ausiliarie progettate per velocizzare il recupero dei record in base a una chiave di ricerca, evitando la scansione sequenziale del file.

Un indice contiene una collezione di **Data Entries** (voci dei dati), denotate come k^* , che permettono di trovare i record con chiave k .

Esistono tre alternative principali per memorizzare le voci k^* all'interno di un indice:

- **Alternativa 1 (Indice Clusterizzato / Index-Organized):** la struttura dell'indice contiene il **record dati completo**.

$$k^* = \langle \text{Intero Record} \rangle$$

In questo caso non c'è un file dati separato; l'indice è il file. La ricerca è rapidissima, ma l'indice diventa molto grande.

- **Alternativa 2 (Indice $\langle K, RID \rangle$):** l'indice contiene la coppia chiave e RID del record.

$$k^* = \langle k, RID \rangle$$

Se ci sono più record con la stessa chiave k (duplicati), avrò più voci distinte nell'indice (es. $\langle k, RID_1 \rangle, \langle k, RID_2 \rangle$).

- **Alternativa 3 (Indice $\langle K, \text{ListaRID} \rangle$):** l'indice contiene la chiave e una lista di tutti i RID che soddisfano quella chiave.

$$k^* = \langle k, (RID_1, RID_2, \dots, RID_n) \rangle$$

Questo formato è più compatto per chiavi con molti duplicati, poiché la chiave k viene memorizzata una volta sola.

6.1 Ricerca per intervallo (Range Search)

Le ricerche per intervallo (ad esempio, `SELECT * WHERE Età > 18`) risultano particolarmente efficienti quando si utilizzano indici ordinati (come gli alberi B+ o le strutture ISAM). Questi indici, infatti, supportano l'accesso sequenziale ai record di dati (le voci k^*) seguendo l'ordine imposto dalla chiave di ricerca, minimizzando i salti sul disco.

6.2 ISAM (Indexed Sequential Access Method)

L'ISAM è una struttura dati ad albero ideata per ottimizzare la gestione di file ordinati tramite un indice primario. Il suo design è fortemente orientato alle prestazioni su disco: i vari nodi dell'albero sono memorizzati in modo contiguo per massimizzare l'efficienza degli accessi sequenziali, rendendo superflui i puntatori fisici tra nodi adiacenti dello stesso livello.

La creazione di un file ISAM avviene per fasi sequenziali:

1. **Allocazione delle pagine dati (foglie):** Vengono scritte su disco in sequenza, riempiendole con i record già ordinati secondo la chiave di ricerca.
2. **Allocazione delle pagine indice:** Vengono generate le pagine non foglia, contenenti coppie del tipo $\langle \text{chiave}, \text{PageID} \rangle$, che fungono da guida per la navigazione verso le pagine dati.
3. **Allocazione dello spazio di overflow:** Viene riservata un'area del disco destinata a ospitare i record delle future operazioni di inserimento.

Operazioni principali

- **Voci di indice (Index entries):** Formattate come $\langle \text{valore chiave}, \text{PageID} \rangle$, indirizzano la ricerca incanalandola verso il sotto-albero che contiene la pagina dati corretta.

- **Ricerca:** Inizia dalla radice e confronta la chiave cercata con quelle presenti nel nodo. Seguendo i puntatori, si scende fino a raggiungere la pagina foglia desiderata. Il costo computazionale è pari a $\mathcal{O}(\log_F N)$, dove F è il *fan-out* (numero di voci per pagina indice) ed N è il numero di pagine dati.
- **Inserimento:** Si individua la pagina dati di destinazione. Se la pagina dispone di spazio libero, il record viene inserito al suo interno; se è piena, il record viene collocato in una **pagina di overflow** concatenata alla foglia originale.
- **Cancellazione:** Si individua e si rimuove il record dalla pagina dati o di overflow in cui risiede. Se, a seguito di cancellazioni, una pagina di overflow si svuota completamente, questa viene deallocata.

Una caratteristica fondamentale dell'ISAM è la sua natura **statica**: una volta inizializzato il file, la struttura dell'albero delle pagine indice non subisce alcuna modifica (non avvengono né split né merge dei nodi). Le operazioni dinamiche (inserimenti e cancellazioni) sono gestite unicamente tramite le pagine dati e le catene di overflow. Questo approccio evita il costo di riarrangiamento dell'albero ma, in presenza di frequenti inserimenti, la crescita eccessiva delle catene di overflow porta a un drastico degrado delle prestazioni. In tal caso, si rende necessaria la ricostruzione periodica dell'intero file.

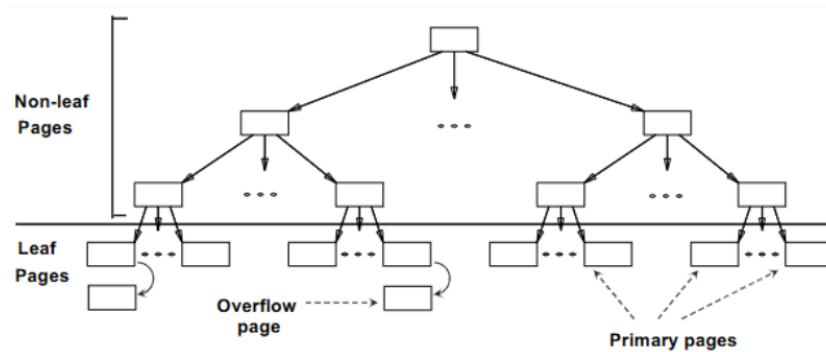


Figure 14: Rappresentazione della struttura di un albero ISAM

Vantaggi e Svantaggi

- **Vantaggi:** Struttura estremamente compatta e assenza di spazio sprecato. È la soluzione ideale per scenari statici o con rari aggiornamenti, poiché l'assenza di operazioni di bilanciamento rende l'accesso in sola lettura molto rapido.
- **Svantaggi:** Forte degrado delle prestazioni in scenari dinamici a causa dell'allungamento delle catene di overflow. Inoltre, presupponendo che il file dati sia ordinato fisicamente, è possibile costruire al massimo un solo indice ISAM primario (o *clustered*) per ogni relazione.

Esempio di inserimento Si consideri l'inserimento sequenziale dei record con chiave di ricerca 23^* , 48^* , 41^* , 42^* . Se le pagine foglia primarie sono piene, questi record andranno a popolare le relative catene di overflow.

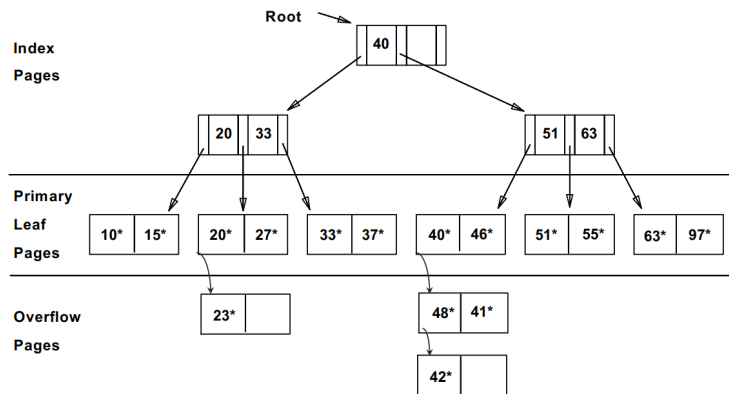


Figure 15: Evoluzione dell'albero ISAM a seguito di inserimenti (con utilizzo di pagine di overflow)

Esempio di cancellazione Consideriamo ora la cancellazione delle chiavi 42*, 51*, 97*.

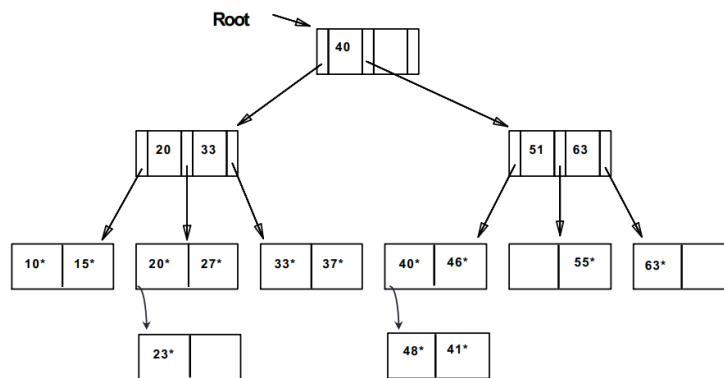


Figure 16: Stato dell'albero ISAM dopo l'operazione di cancellazione

Osservando la figura, è possibile notare che se la chiave 51* fungeva da separatore, essa **continuerà ad apparire nelle pagine indice** anche dopo essere stata rimossa dalle foglie. Questo è un effetto diretto della natura statica dell'ISAM, che non aggiorna la struttura di routing.

Inoltre, la gestione dello spazio vuoto differisce in base al tipo di pagina: se si cancella un record (ad esempio 41*) da una pagina di overflow, i record successivi (come 42*) vengono compattati traslando verso l'alto, senza lasciare "buchi". Al contrario, nelle pagine foglia primarie, la rimozione di un record lascia tipicamente uno spazio vuoto non ricompattato.

6.3 Indici B+ Tree

I B+ Tree sono strutture dati gerarchiche, dinamiche e bilanciate, ideate specificamente per superare le rigidità strutturali tipiche dell'ISAM. A differenza di quest'ultimo, i B+ Tree si adattano in modo dinamico ai cambiamenti dei dati, mantenendo un'efficienza costante senza la necessità di ricalcolare l'intera struttura o affidarsi a catene di overflow.

Come gli ISAM, i B+ Tree sono organizzati in livelli di nodi che hanno dimensione pari a quella di una pagina su disco. Un'altra similarità è il fatto che entrambi sono organizzati in strutture gerarchiche.

Nei B+ Tree, il file può essere come non ordinato, ma non è necessario che lo sia, poiché tutte le chiavi di ricerca vengono replicate e ordinate a livello delle foglie. Se il file non è ordinato devo avere un livello soprastante di foglie dove sono presenti tutte le chiavi di ricerca, in modo ordinato, ciascuno con il puntatore al record reale, perché non potrei sfruttare il maggiore e minore per navigare l'albero.

Aggiungere un esempio di b tree con record non ordinati

Struttura e Caratteristiche

- **Nodi interni (Pagine indice):** Contengono esclusivamente voci di routing (chiavi di separazione e puntatori) per indirizzare la ricerca verso il sotto-albero corretto. Non contengono i dati effettivi.
- **Nodi foglia (Pagine dati):** Contengono le voci di dato (o i puntatori ai record reali) e sono mantenuti rigorosamente ordinati in base alla chiave di ricerca.
- **Concatenazione delle foglie (Sequence Set):** Poiché nei B+ Tree i nodi dello stesso livello **non sono allocati contigualmente** su disco (vengono salvati dove c'è spazio), le pagine foglia sono collegate tra loro tramite puntatori (*next* e *previous*), formando una lista doppiamente concatenata. Questo garantisce un accesso sequenziale estremamente efficiente. Questi puntatori sono trasparenti all'utente e vengono gestiti dal sistema.
- **Indipendenza dall'ordinamento fisico:** Non è richiesto che il data file sottostante sia fisicamente ordinato. Questo permette di definire molteplici indici (B+ Tree) sulla stessa tabella per diverse chiavi di ricerca (*unclustered index*). Tutte le chiavi vengono infatti replicate e ordinate a livello delle foglie.
- **Vincolo di riempimento (Occupancy):** Per mantenere l'albero bilanciato e le prestazioni prevedibili, ogni nodo (fatta eccezione per la radice) deve mantenere un fattore di riempimento compreso tra il 50% e il 100% della sua capienza massima. Viene deliberatamente lasciato dello spazio libero per assorbire futuri inserimenti limitando le modifiche all'architettura dell'albero.

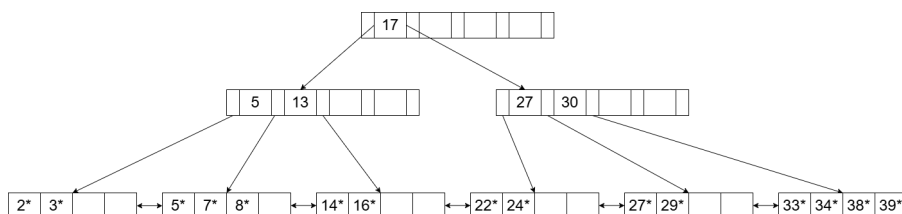


Figure 17: Rappresentazione della struttura e dei collegamenti a livello foglia in un albero B+ Tree

Gestione Dinamica: Inserimento e Cancellazione La forte dinamicità del B+ Tree si esprime nella sua capacità di autobilanciarsi a seguito di operazioni di aggiornamento:

- **Inserimento (Split):** Si individua la foglia corretta. Se il nodo è pieno, avviene uno *split*: il nodo viene diviso in due, i dati vengono equamente distribuiti e la chiave mediana viene **copiata** (se si divide una foglia) o **promossa** (se si divide un nodo interno) al nodo padre. Se anche il padre risulta pieno, l'operazione di split si propaga verso l'alto (*cascade*). Se lo split raggiunge la radice e la divide, viene creata una nuova radice, incrementando l'altezza dell'albero di un livello.
- **Cancellazione (Merge e Redistribuzione):** Si rimuove l'elemento dalla foglia. Se il livello di riempimento del nodo scende al di sotto del 50%, si viola il vincolo strutturale. L'albero tenta prima una **redistribuzione**, prendendo in prestito elementi da un nodo "fratello" adiacente che ha spazio in abbondanza. Se i nodi adiacenti sono a loro volta al limite minimo del 50%, si procede con un **merge** (fusione): i due nodi vengono accorpati e la chiave di routing associata nel padre viene eliminata. Anche il merge può propagarsi a cascata verso la radice, arrivando potenzialmente a ridurre l'altezza complessiva dell'albero.

Questa operazione è molto costosa, quindi si cerca di rimandarla il più possibile sperando che arrivino nuovi inserimenti che riempiano i nodi prima di doverli unire.

6.3.1 Impatto della Dimensione della Chiave sulle Prestazioni

La dimensione in byte della chiave di ricerca ha un impatto diretto e profondo sulle prestazioni complessive dell'albero B+ Tree, poiché determina il **fan-out** (il numero massimo di figli per nodo).

- **Chiavi semplici (es. Interi):** Occupano pochi byte. Questo permette di inserire un numero elevato di voci all'interno di una singola pagina (fan-out elevato). Un fan-out maggiore produce un albero molto "largo" e **basso**, riducendo al minimo il numero di accessi al disco (I/O) necessari per raggiungere le foglie partendo dalla radice.

- **Chiavi complesse (es. Stringhe lunghe o chiavi composte):** Occupano molto spazio. Il numero di voci per pagina diminuisce drasticamente (fan-out ridotto), costringendo l'albero a crescere in **altezza**. Ogni livello aggiuntivo dell'albero si traduce in una potenziale operazione di I/O meccanica in più durante la ricerca, degradando le prestazioni.

Il Grado dell'Albero (d) La capacità strutturale di un nodo è formalmente definita dal parametro d , chiamato **grado dell'albero**. In un B+ Tree, per garantire il bilanciamento dinamico e l'efficienza dello spazio, ogni nodo (ad eccezione della radice) deve rispettare un vincolo di riempimento minimo e massimo:

$$d \leq m \leq 2d$$

dove m è il numero di chiavi di ricerca attualmente presenti nel nodo. Di conseguenza, un nodo interno che contiene m chiavi avrà esattamente $m + 1$ puntatori ai sottonodi figli.

6.3.2 Compressione delle chiavi basata su prefissi (Prefix Key Compression)

Per mitigare il problema dimensionale delle chiavi complesse e massimizzare il fan-out dei nodi interni, si ricorre a tecniche di compressione. L'obiettivo è memorizzare all'interno delle pagine indice solo l'informazione strettamente necessaria per instradare la ricerca verso la foglia corretta, scartando i dati ridondanti.

Questa tecnica si rivela particolarmente efficace quando le chiavi di ricerca sono **stringhe testuali** (es. VARCHAR), sfruttando l'idea di memorizzare solo il **prefisso discriminante minimo**.

Funzionamento: Invece di copiare l'intera chiave per separare due sotto-alberi, il DBMS calcola e memorizza il prefisso più corto che sia strettamente sufficiente a distinguere il valore massimo del sotto-albero sinistro dal valore minimo del sotto-albero destro.

Esempio pratico di discriminazione Supponiamo che il DBMS debba generare una chiave di instradamento (separatore) in un nodo interno per dividere i record che iniziano per **Pa11a** da quelli che iniziano per **Pa11one**.

- **Senza compressione:** La chiave di separazione nel nodo interno occuperebbe 7 byte (**Pa11one**).
- **Con compressione:** È sufficiente utilizzare il prefisso **Pa11o** (5 byte). Durante la discesa dell'albero, il sistema sa che tutto ciò che è $< \text{Pa11o}$ va a sinistra (trovando **Pa11a**), e tutto ciò che è $\geq \text{Pa11o}$ va a destra (trovando **Pa11one**).
- **Gestione della dinamicità:** Se in futuro inseriamo il record **Pa11ottola** e questo causa uno split generando una nuova pagina, il vecchio separatore non sarà più sufficiente. Per discriminare tra la pagina di **Pa11one** e quella di **Pa11ottola**, il DBMS dovrà ricalcolare dinamicamente la chiave di routing allungandola a **Pa11ot** (poiché $\text{Pa11on} < \text{Pa11ot} \leq \text{Pa11ottola}$).

Vantaggi, Svantaggi e Trade-off L'adozione di questa tecnica richiede un'attenta valutazione architetturale:

- **Vantaggi (Spazio e I/O):** Aumenta significativamente la densità di chiavi per pagina indice. Questo mantiene l'albero più basso e riduce drasticamente i colli di bottiglia legati all'accesso al disco.
- **Svantaggi (Overhead CPU e Manutenzione):** La compressione introduce un costo computazionale per il processore (CPU), che deve codificare e decodificare le chiavi al volo durante l'attraversamento. Inoltre, in scenari altamente dinamici, il continuo ricalcolo dei prefissi a seguito degli *split* comporta un pesante degrado delle prestazioni in scrittura.
- **Conclusione:** È una soluzione fortemente raccomandata per scenari **statici** o *read-mostly* (es. cataloghi storici, Data Warehouse), mentre la sua convenienza nei sistemi transazionali (OLTP) ricchi di INSERT e DELETE va misurata caso per caso in base all'effettivo risparmio di byte.

6.3.3 Inizializzazione di un B+ Tree: Bulk Loading

Costruire un B+ Tree inserendo i record uno ad uno (inserimento standard) può diventare un processo estremamente costoso dal punto di vista computazionale e delle operazioni di I/O, a causa dei continui attraversamenti dell'albero dall'alto verso il basso e delle frequenti operazioni di *split* e *merge* dei nodi.

Quando si deve popolare un indice con una grande mole di dati preesistenti, l'approccio ottimale è il **Bulk Loading** (caricamento massivo). Invece di modificare continuamente i puntatori in modo casuale, questo algoritmo costruisce l'albero "dal basso verso l'alto" (bottom-up) sfruttando l'ordinamento.

Il procedimento si articola in tre fasi principali:

1. **Ordinamento:** Si estraggono le *Data Entries* (k^*) e si ordinano in base alla chiave di ricerca (ad esempio, tramite un *External Merge Sort*).
2. **Creazione delle foglie:** Si allocano le pagine foglia vuote e si riempiono sequenzialmente con i dati già ordinati. Poiché i dati arrivano in ordine crescente, l'algoritmo riempie una pagina fino a una determinata **soglia di riempimento** (*Fill Factor*, es. 80%, per lasciare spazio a futuri inserimenti organici) e poi passa a una nuova pagina, modificando solo il puntatore della foglia più a destra.
3. **Costruzione dell'indice:** Man mano che le foglie vengono riempite, il sistema estrae il valore minimo della nuova pagina (chiave di separazione) e lo inserisce nel nodo padre al livello superiore. Il processo si ripete ricorsivamente verso l'alto fino a generare un'unica pagina radice.

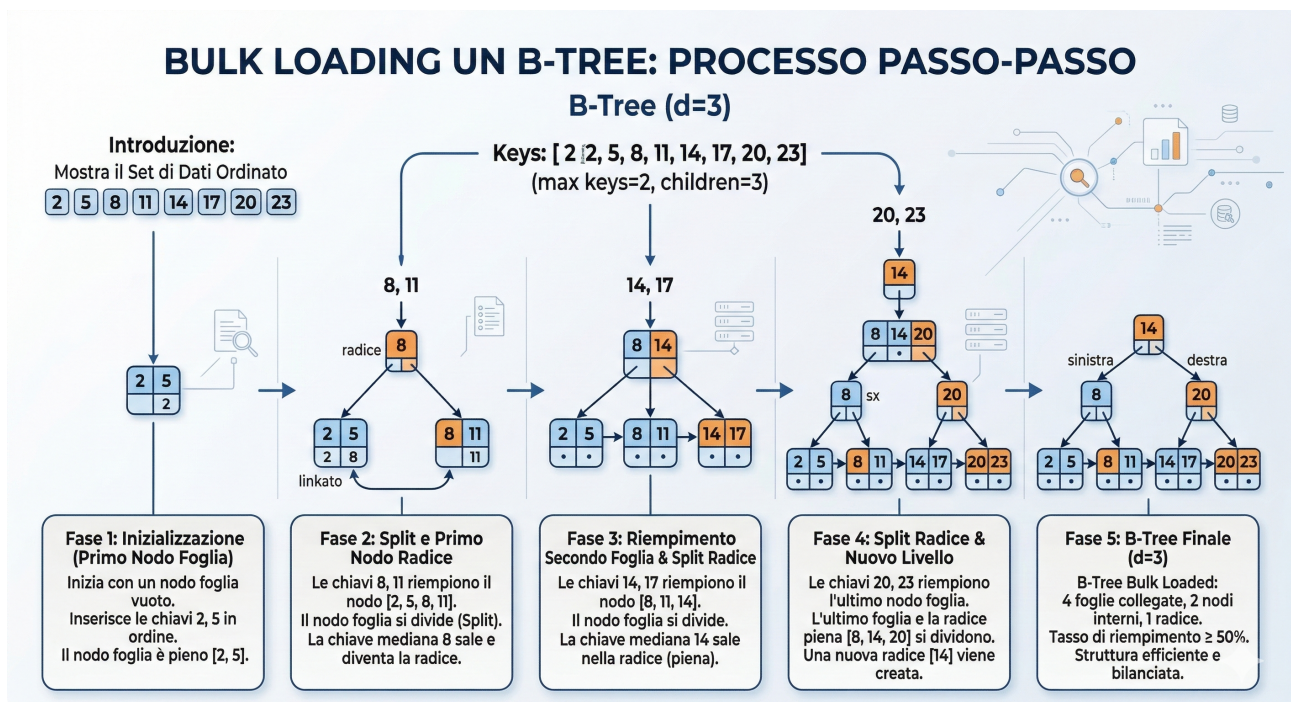


Figure 18: Esempio di bulk loading: costruzione bottom-up partendo dai dati pre-ordinati.

6.4 Classificazione degli indici

Gli indici possono essere classificati in base al loro ruolo logico rispetto allo schema della base di dati:

- **Indice Primario (Primary Index):** È l'indice costruito sulla **chiave primaria** della relazione.
 - Essendo la chiave primaria univoca per definizione, questo indice non contiene chiavi duplicate.
 - Tipicamente, i DBMS relazionali creano automaticamente questo indice di default alla definizione del vincolo PRIMARY KEY, in quanto è il campo più frequentemente utilizzato nelle operazioni di ricerca esatta.
- **Indice Secondario (Secondary Index):** È un indice ausiliario costruito su uno o più attributi che **non** costituiscono la chiave primaria (es. "Cognome", "Dipartimento").

- A differenza di quello primario, può contenere valori duplicati (a meno che non sia imposto un vincolo **UNIQUE**).
- È fondamentale per ottimizzare le query di ricerca che filtrano o ordinano i dati in base a campi secondari.

6.5 Indici Clusterizzati e Non Clusterizzati

Una classificazione critica per le prestazioni fisiche (costo di I/O) riguarda la correlazione tra l'ordine logico dell'indice e l'ordine fisico dei record nel file dati su disco.

- **Indice Clusterizzato (*Clustered Index*):** In un indice clusterizzato, l'**ordine fisico dei record** all'interno del file dati corrisponde (o è fortemente correlato) all'ordine logico della chiave di ricerca nell'indice.
 - **Gestione delle pagine:** Si parla di dati "quasi ordinati" perché, sebbene all'interno di una singola pagina i record possano subire lievi frammentazioni, le pagine nel loro complesso sono ordinate sequenzialmente tra di loro.
 - **Efficienza per Intervalli:** Garantisce prestazioni eccezionali per le **ricerche per intervallo** (*range query*). Scandendo le chiavi ordinate nelle foglie, i puntatori accedono a blocchi fisicamente contigui su disco. Visivamente, i puntatori "non si incrociano".
 - **Page Faults Minimi:** Una volta caricata in RAM la pagina per il primo record dell'intervallo, i successivi si troveranno molto probabilmente nella stessa pagina, ammortizzando drasticamente i costi di I/O (la pagina viene portata in memoria una sola volta).
 - **Unicità del Clustered:** Poiché i dati fisici non possono essere ordinati in più modi contemporaneamente, **può esistere al massimo un solo indice clusterizzato per tabella**.

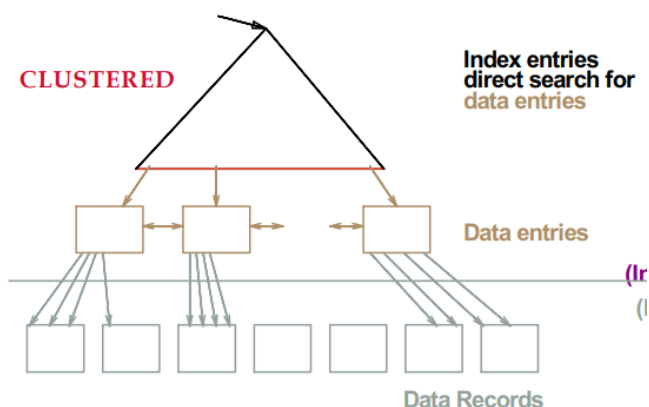


Figure 19: Esempio di indice clusterizzato: i puntatori fluiscono parallelamente verso i blocchi fisici.

- **Indice Non Clusterizzato (*Unclustered Index*):** Non vi è alcun vincolo o correlazione tra l'ordine logico delle chiavi nell'indice e l'organizzazione fisica dei record nel file dati (che tipicamente risiede in un *Heap File* o è clusterizzato su un'altra chiave).
 - **Costo di Scansione:** Una scansione sequenziale delle foglie dell'indice comporta l'inseguimento di puntatori che saltano casualmente da un blocco fisico all'altro sul disco. Visivamente, i puntatori "si incrociano" generando caos.
 - **Collasso delle Prestazioni (Page Faults):** Nel caso peggiore, l'accesso sequenziale a N chiavi nell'indice può provocare il caricamento di N pagine disco differenti. Questo rende gli indici non clusterizzati molto inefficienti per ampi intervalli di ricerca.
 - **Molteplicità:** Essendo slegati dall'ordine fisico, è possibile definire **molteplici indici non clusterizzati** sulla stessa tabella.

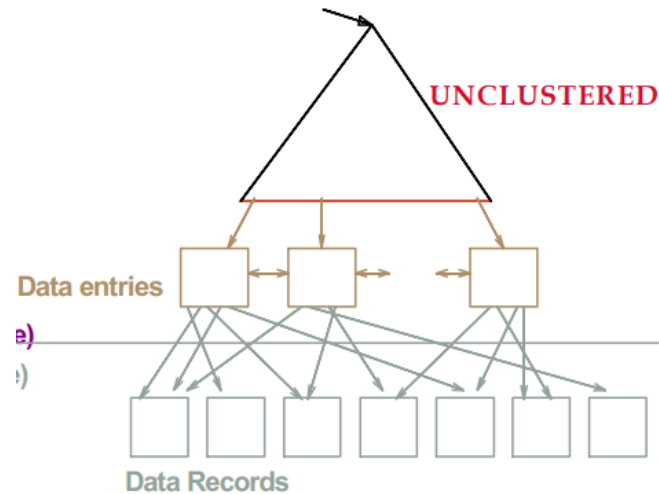


Figure 20: Esempio di indice non clusterizzato: incrocio dei puntatori e conseguente elevato costo di accesso casuale (Random I/O).

6.5.1 Indice Clusterizzato (Clustered Index)

In un indice clusterizzato, l'ordine fisico dei record all'interno del file dati corrisponde all'ordine logico della chiave di ricerca nell'indice.

A livello fisico, l'ordinamento rigoroso dei record all'interno della singola pagina dati non è strettamente necessario; ciò che è fondamentale è l'ordinamento sequenziale tra le pagine. Una volta caricata la pagina in RAM, infatti, il costo per esplorarla e trovare il record internamente è trascurabile.

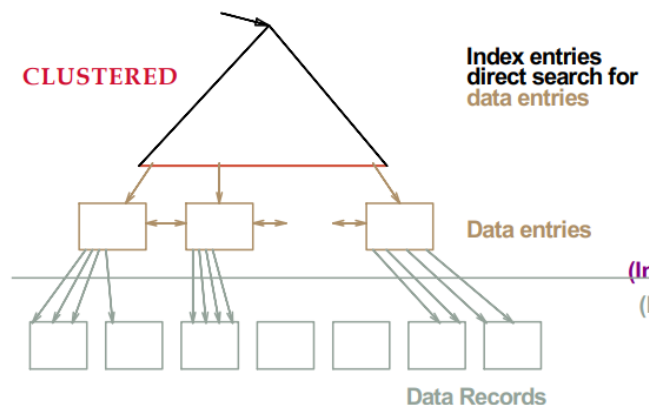


Figure 21: Esempio di indice clusterizzato: i puntatori fluiscono parallelamente verso i blocchi fisici.

Vantaggi

- **Efficienza per Intervalli (Range Query):** Garantisce prestazioni eccellenti per le ricerche di intervalli. Scandendo le chiavi nelle foglie, i puntatori accedono a blocchi fisicamente contigui, riducendo al minimo i salti sul disco.
- **Compressione strutturale:** Facendo leva sull'ordinamento fisico, è possibile mantenere a livello foglia solo gli *estremi degli intervalli* (costruendo un indice sparso). Questo riduce il numero di chiavi necessarie nelle foglie e, di conseguenza, il numero di pagine dell'indice. Il risultato è un albero più piccolo, meno profondo e con tempi di attraversamento più rapidi.

Svantaggi

- **Unicità:** Poiché i dati fisici non possono essere ordinati in più modi contemporaneamente, può esistere un solo indice clusterizzato per tabella.

- **Accesso ai Data File:** Se l'obiettivo è restituire solo i valori della chiave di ricerca, in un indice sparso è comunque necessario accedere alle pagine dei data file, poiché non tutte le chiavi sono esplicitamente rappresentate nelle foglie dell'indice.

6.5.2 Indice Non Clusterizzato (*Unclustered Index*)

Non vi è alcuna correlazione tra l'ordine logico delle chiavi nell'indice e l'organizzazione fisica dei record nel file dati. A causa di questa disconnessione, l'indice **deve** essere denso: tutte le chiavi di ricerca devono essere obbligatoriamente presenti e ordinate a livello delle foglie.

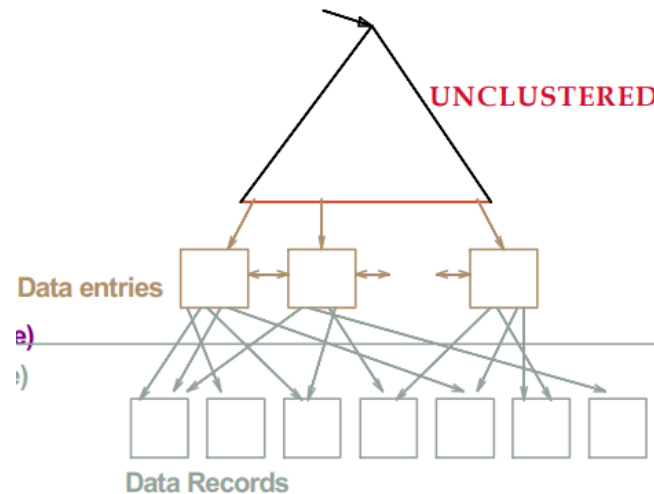


Figure 22: Esempio di indice non clusterizzato: incrocio dei puntatori e conseguente elevato costo di accesso casuale (Random I/O).

Vantaggi

- **Index-Only Query altamente efficienti:** Se la query richiede di restituire solo le chiavi di ordinamento, il DBMS può fermarsi al livello delle foglie dell'indice. Essendo le foglie molto più piccole rispetto ai data record completi, il numero di accessi in lettura è drasticamente inferiore.
- **Molteplicità:** Non essendo vincolati all'ordinamento fisico, è possibile definire molteplici indici non clusterizzati sulla stessa tabella.

Svantaggi

- **Collasso in scansione completa:** Se la query non è *index-only* e richiede l'intero record, l'operazione diventa estremamente costosa. Il DBMS deve posizionarsi sulla foglia e seguire i puntatori, generando accessi casuali al disco. Nel caso peggiore, il numero di page fault è pari al numero di record che soddisfano la query.

6.5.3 Casi di Studio: Domande d'Esame

Domanda 1: Se si sa che la query più frequente sarà la ricerca del valore massimo dell'attributo su cui è costruito l'indice, è preferibile un indice clusterizzato o non clusterizzato?

Risposta: Sono **equivalenti** in termini di prestazioni. Nell'unclustered, si sa che il valore massimo si trova nell'ultima voce dell'ultima foglia. Nel clustered, si segue la medesima struttura fino ad arrivare all'ultima pagina dati (che va portata in RAM per leggerne il massimo). Il numero di letture strutturali (l'altezza dell'albero) è sostanzialmente lo stesso.

Domanda 2: E se la query fosse il calcolo del valore medio (AVG) sull'attributo indicizzato?

Risposta: È nettamente preferibile l'**indice non clusterizzato**. Per calcolare la media è sufficiente leggere i soli valori della chiave. Con l'unclustered si esegue un'efficiente *index-only query* leggendo esclusivamente le foglie dell'indice. Con il clustered (essendo spesso sparso), si è costretti a scandire

tutte le N pagine dell'intero data file per estrarre i valori, comportando un costo di I/O molto più elevato.

Domanda 3: Se si dovesse stampare ordinatamente i valori di un attributo che NON è la chiave di ricerca su cui è costruito l'indice, è meglio un *clustered* o un *unclustered*?

Risposta: Meglio il **clusterizzato**. Dovendo in ogni caso scandire l'intera relazione (poiché l'indice esistente non aiuta a filtrare o ordinare quell'attributo specifico), il file clusterizzato garantisce l'accesso sequenziale ai blocchi fisici. Usare un indice *unclustered* per accedere sequenzialmente a tutti i dati genererebbe un numero inaccettabile di accessi casuali.

6.5.4 Gestione dei Duplicati e Struttura delle Voci (Data Entries)

La presenza di chiavi di ricerca duplicate (più record con la stessa chiave) influisce su come le Data Entries (k^*) vengono formattate nelle foglie dell'indice:

- **Alternativa 1** ($k^* = \langle \text{Intero Record} \rangle$): Non si pone il problema dei puntatori duplicati, poiché le foglie ospitano direttamente i record. Eventuali record con la stessa chiave di ricerca differiranno inevitabilmente per la chiave primaria. Questa alternativa ha senso unicamente per gli indici clusterizzati.

- **Alternativa 2** ($k^* = \langle k, RID \rangle$): Se esistono 5 record con la stessa chiave, si avranno 5 chiavi ripetute e 5 puntatori distinti all'interno dell'indice.

Vantaggio: Garantisce record a *lunghezza fissa*, rendendo la gestione dello spazio nella pagina estremamente semplice e veloce.

Svantaggio: Maggiore occupazione di spazio a causa della ridondanza della chiave.

- **Alternativa 3** ($k^* = \langle k, \text{ListaRID} \rangle$): La chiave compare una singola volta, abbinata alla lista di tutti i puntatori che la soddisfano.

Vantaggio: Ottimizzazione e forte risparmio di spazio.

Svantaggio: Introduce record a *lunghezza variabile*. La gestione strutturale (allocazione, aggiornamenti, frammentazione) risulta molto più complessa da implementare nel DBMS.

6.6 Indici con chiavi di ricerca composte

È possibile definire un indice utilizzando un numero arbitrario di attributi, creando così una **chiave di ricerca composta** (o *composite key*).

Nei database, l'ordinamento lessicografico di una chiave composta è rigorosamente sequenziale e posizionale. Di conseguenza, indicizzare per (Cognome, Nome) è radicalmente diverso dall'indicizzare per (Nome, Cognome): nel primo caso, il DBMS ordina l'intero set di dati primariamente per Cognome e, solo in caso di omonimia (duplicati), utilizza il Nome come criterio di spartizione per l'ordinamento secondario.

Quando utilizzare chiavi composte L'indicizzazione composita diventa cruciale quando l'analisi del *query workload* (il carico di lavoro previsto sul database) rivela la presenza frequente di query che filtrano i dati su più attributi contemporaneamente nella clausola WHERE.

Un esempio tipico è: `SELECT * FROM Studenti WHERE Cognome = 'Rossi' AND AnnoNascita > 2002 AND AnnoNascita < 2006;`

Attenzione al Trade-off: Ogni indice aggiuntivo o complesso impone un onere (*overhead*) sulle operazioni di scrittura (INSERT, UPDATE, DELETE), poiché il DBMS deve aggiornare la struttura dell'albero per ogni modifica. Pertanto, la creazione di indici composti è giustificata solo se i benefici in fase di lettura superano i costi di manutenzione.

6.6.1 Strategie di esecuzione per query multi-attributo

Di fronte alla query d'esempio, il DBMS può adottare diverse strategie (o *piani di esecuzione*) a seconda degli indici effettivamente disponibili:

- **Scenario 1: Indice singolo solo su *Cognome***
 - L'albero naviga rapidamente fino a trovare il primo record con cognome 'Rossi'.
 - Segue sequenzialmente i puntatori per recuperare *tutti* i record dei 'Rossi' dai file dati.
 - Solo dopo averli caricati in RAM, verifica l'anno di nascita, scartando quelli fuori dal range. Il costo dipenderà da quanti 'Rossi' esistono in totale (selettività del cognome).
- **Scenario 2: Indice singolo solo su *AnnoNascita***
 - Effettua una ricerca per intervallo (Range Query) estraendo i record degli anni 2003, 2004 e 2005.
 - Accede ai file dati per tutti questi studenti e verifica se il cognome è 'Rossi'.
 - Questa opzione è tipicamente inefficiente: essendo i range di età poco selettivi, il sistema finirà per caricare un'enorme mole di dati inutili solo per trovare pochi 'Rossi'.
- **Scenario 3: Due indici singoli separati (*Index Intersection*)**
 - Sfrutta entrambi gli indici in modo indipendente: estrae dall'indice dei cognomi la lista dei RID (puntatori) di tutti i 'Rossi' e, dall'indice delle date, la lista dei RID di chi è nato nel range.
 - Il DBMS calcola in memoria l'**intersezione** di questi due insiemi di puntatori.
 - Infine, accede ai file dati *esclusivamente* per recuperare i record che appartengono all'intersezione, azzerando le letture inutili.
- **Scenario 4: Indice Composto (La soluzione ottimale)** Se la query è molto frequente, un indice composto è la scelta vincente. Tuttavia, **l'ordine degli attributi nell'indice è fondamentale**:
 - **Opzione A - Ordinamento** (*AnnoNascita*, *Cognome*): L'attributo primario è soggetto a un range. L'indice individua il primo record > 2002 . Da quel punto in poi, il DBMS è costretto a **scandire sequenzialmente tutto il blocco** di nati nel 2003, 2004 e 2005, saltando continuamente i record con cognomi diversi da 'Rossi'. Molto costoso.
 - **Opzione B - Ordinamento** (*Cognome*, *AnnoNascita*): L'attributo primario è soggetto a un'uguaglianza. L'indice salta direttamente al blocco esatto di tutti i 'Rossi' e, all'interno di questo sottoinsieme già isolato, si posiziona sull'estremo inferiore del range ('Rossi', 2003). Da lì scorre sequenzialmente leggendo **solo record validi**, fino a quando l'anno non supera il 2005.

Regola d'Oro del Design degli Indici Dall'ultimo scenario emerge una regola fondamentale per la progettazione degli indici composti: **Equality First, Range Second** (Prima le uguaglianze, poi gli intervalli).

Se un *query workload* prevede l'uso combinato di predicati di uguaglianza (es. =) e predicati di intervallo (es. <, >, BETWEEN), l'indice composto deve **sempre** avere l'attributo di uguaglianza come chiave primaria e l'attributo di intervallo come chiave secondaria. In questo modo si massimizza la precisione della discesa nell'albero e si minimizzano i record letti scartati.

6.7 Indici Hash-Based

Gli indici basati su tabelle di **Hash** sono strutture progettate per massimizzare l'efficienza delle **ricerche per uguaglianza esatta** (es. SELECT * FROM Impiegati WHERE Matricola = 12345).

A differenza degli alberi B+ Tree, le funzioni di hash mirano per loro stessa natura a distribuire i record in modo uniforme e pseudo-casuale su tutto lo spazio degli indirizzi disponibili. Questo comporta una **totale assenza di correlazione** tra l'ordine logico delle chiavi di ricerca e l'ordine fisico dei record su disco.

Caratteristiche Principali

- **Accesso in tempo costante:** Nel caso ideale (assenza di collisioni), per recuperare la pagina contenente il record desiderato è sufficiente **un singolo accesso al disco** (I/O), applicando semplicemente la funzione di hash $h(k)$ alla chiave di ricerca k .
- **Inadeguatezza per Range Query:** Proprio a causa della distribuzione disordinata che distrugge qualsiasi ordinamento logico, gli indici hash **non possono** essere utilizzati per risolvere in modo efficiente query per intervallo (es. `WHERE Età > 18`). Per queste operazioni, il file dovrebbe essere scandito per intero, rendendo il B+ Tree la scelta obbligata.

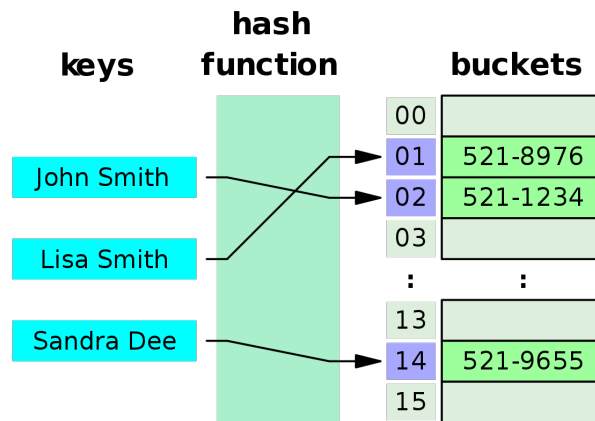


Figure 23: Esempio di hashing

Livelli di Applicazione dell'Hashing L'indicizzazione hash può essere implementata in due modalità architetturali:

1. **Hashing diretto sul Data File:** La funzione di hash calcola l'indirizzo fisico in cui posizionare direttamente il record dati completo (corrispondente all'*Alternativa 1*). Poiché i record fisici possono essere disposti in un solo modo su disco, è possibile avere **un solo** indice di questo tipo per tabella.
2. **Hashing sulle Data Entries:** La funzione di hash organizza esclusivamente le voci di indice (es. *Alternativa 2* o *Alternativa 3*), le quali fungono da directory contenente le chiavi e i puntatori (RID) alla posizione reale dei record nei data file (ad esempio, un *Heap File*). Essendo svincolato dall'ordinamento fisico dei dati veri e propri, questo approccio permette di costruire **molteplici indici hash** secondari sulla stessa tabella.

6.7.1 Hashing Statico

L'hashing statico è la forma architetturale più semplice. Si basa sull'allocazione di un numero **prefissato e immutabile di bucket** (pagine primarie) fin dal momento della creazione del file.

Inizializzazione: Poiché non è possibile conoscere a priori l'esatta distribuzione e il volume dei futuri inserimenti, il DBMS (o il DBA) deve stimare il numero totale di record. Tipicamente si dimensiona la struttura per avere un fattore di riempimento (*Fill Factor*) iniziale ragionevole (es. 80%), lasciando dello spazio vuoto fisiologico per assorbire futuri inserimenti organici.

Il Problema del Degrado: Collisioni e Overflow Il principale punto debole dell'hashing statico è la sua rigidità strutturale di fronte alla crescita dei dati. Se il database subisce numerosi inserimenti non previsti o se la funzione di hash distribuisce i dati in modo non omogeneo (fenomeno dello *skew*), i bucket primari si riempiono rapidamente, generando **collisioni**.

Quando un bucket si satura e non c'è più spazio per il record k^* , il DBMS è costretto ad allocare **pagine di overflow** concatenate dinamicamente al bucket originale. L'impatto sulle prestazioni è severo:

- L'efficienza ideale di un singolo accesso $\mathcal{O}(1)$ **degrada rapidamente** in una **lettura sequenziale** (scansione) di tutte le pagine che compongono la catena di overflow.
- In scenari pessimi (catene molto lunghe), il costo per trovare un record supera quello di un attraversamento di un albero B+ Tree.
- Per ripristinare le prestazioni originali, si rende necessaria una costosa **riorganizzazione periodica** dell'intero file (re-hashing), che richiede di fermare le operazioni, definire una nuova funzione di hash e ridistribuire tutti i dati su un set di bucket più ampio.